Topic 1. Location Services

1.1 Introduction to location services

Definition: Location services are technologies that allow a mobile application to determine the geographical position of a device and use that information to provide location-aware features.

A location is commonly represented by two coordinates:

- Latitude: The angular position north or south of the Equator.
- Longitude: The angular position east or west of the Prime Meridian.

For example, an application might receive a location represented by latitude 13.0827 and longitude 80.2707, approximately corresponding to Chennai.

Why are location services needed?

Without location services, a user would have to manually enter their location into every application. Location-aware applications can instead identify the user's position, show nearby places, calculate journeys and provide relevant information.

Real-world applications

- Google Maps: navigation and nearby places.
- Uber: matching passengers and drivers.
- Swiggy: delivery addresses and order tracking.
- Weather applications: local weather forecasts.
- Fitness applications: running distance and route tracking.
- Emergency applications: sharing a location with trusted contacts.

1.2 How location services work

Physical device

GPS/GNSS · Wi-Fi · Cellular networks

Android location framework

Collects and processes location information

Location API

FusedLocationProviderClient

Android application

Displays coordinates, finds nearby places or tracks a route

The application requests permission before accessing protected location information. The Android location system determines an available location estimate and returns it to the application.

Different location sources

Source	How it works	Typical characteristics
GPS/GNSS	Uses satellite signals	Often accurate outdoors; may struggle indoors
Wi-Fi	Uses nearby Wi-Fi information	Useful in urban and indoor environments
Cellular	Uses mobile network information	Broad coverage; accuracy varies
Fused location	Combines available sources	Convenient balance of accuracy and power use

Important distinction: GPS is a positioning technology. The fused location provider is an API that intelligently combines available location information; it does not mean every request uses GPS.

1.3 Geolocation and FusedLocationProviderClient

What is geolocation?

Geolocation is the process of estimating the geographical position of a device, person or object. In Android, applications can use Google's fused location provider through FusedLocationProviderClient.

Why use FusedLocationProviderClient?

It provides a convenient way to request locations without manually managing GPS, Wi-Fi and cellular positioning separately.

It supports:

- Obtaining a last-known location.
- Requesting a current location.
- Receiving repeated location updates.
- Configuring desired accuracy and update intervals.

Implementation in Android Studio using Java

Step 1 — Add the dependency

In the app-level build.gradle.kts file:

```
dependencies {
    implementation("com.google.android.gms:play-services-location:<version>")
}
```

Replace <version> with the current stable version compatible with your project.

Step 2 — Declare permissions

In AndroidManifest.xml, directly inside the <manifest> element and outside <application>:

```
<uses-permission
    android:name="android.permission.ACCESS_COARSE_LOCATION" />

<uses-permission
    android:name="android.permission.ACCESS_FINE_LOCATION" />
```

- ACCESS_COARSE_LOCATION permits approximate location access.
- ACCESS_FINE_LOCATION permits precise location access when granted.

These declarations alone do not grant permission. On modern Android versions, the application must also request location permission at runtime.

Step 3 — Create the layout

In res/layout/activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp">

    <Button
```

```
    android:id="@+id/btnLocation"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Get My Location" />

  <TextView
    android:id="@+id/txtLocation"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:padding="16dp"
    android:text="Location will appear here" />

</LinearLayout>
```

Step 4 — Request permission and retrieve location

In MainActivity.java:

```
public class MainActivity extends AppCompatActivity {

  private FusedLocationProviderClient client;
  private TextView txtLocation;

  private final ActivityResultLauncher<String[]> locationPermission =
    registerForActivityResult(
      new ActivityResultContracts.RequestMultiplePermissions(),
      permissions -> {
        boolean granted = Boolean.TRUE.equals(
          permissions.get(Manifest.permission.ACCESS_FINE_LOCATION))
          || Boolean.TRUE.equals(
          permissions.get(Manifest.permission.ACCESS_COARSE_LOCATION));
```

```
        if (granted) {
            getLocation();
        } else {
            txtLocation.setText("Location permission denied.");
        }
    }
  );

@Override
protected void onCreate(Bundle savedInstanceState) {
  super.onCreate(savedInstanceState);
  setContentView(R.layout.activity_main);

  client = LocationServices.getFusedLocationProviderClient(this);
  txtLocation = findViewById(R.id.txtLocation);

  findViewById(R.id.btnLocation).setOnClickListener(v ->
    locationPermission.launch(new String[] {
      Manifest.permission.ACCESS_FINE_LOCATION,
      Manifest.permission.ACCESS_COARSE_LOCATION
    })
  );
}

@SuppressLint("MissingPermission")
private void getLocation() {
  // This method is called only after permission is granted.
```

```
        client.getLastLocation().addOnSuccessListener(this, location -> {
            if (location != null) {
                txtLocation.setText(
                    "Latitude: " + location.getLatitude()
                    + "\nLongitude: " + location.getLongitude()
                );
            } else {
                txtLocation.setText(
                    "No cached location. Try requesting a current location."
                );
            }
        });
    }
}
```

Required imports include Manifest, Bundle, TextView, AppCompatActivity, ActivityResultLauncher, ActivityResultContracts, FusedLocationProviderClient and LocationServices.

The example illustrates the permission flow and last-known location. In a production app, handle permission state more completely, including the user's approximate-versus-precise choice, and request a current location if a cached one is unavailable or too old.

Explanation of the important lines

- LocationServices.getFusedLocationProviderClient(this) creates the client used to access location services.
- registerForActivityResult(...) handles the permission request result.
- getLastLocation() retrieves a cached location, if available.
- getLatitude() and getLongitude() extract the coordinates.
- The location != null check prevents the application from trying to read coordinates from a missing result.

Limitations and best practices

- Location can be unavailable or outdated.
- Users can deny permission or choose approximate location.
- Continuous tracking consumes power and needs careful lifecycle management.
- Request only the accuracy and access level needed for the feature.
- Background location access has additional Android restrictions and should only be requested when essential.

1.4 Google Maps SDK for Android

What is it?

The Google Maps SDK lets developers display and control an interactive Google Map inside an Android application.

Geolocation answers “Where is the device?” Google Maps helps answer “Where is that position on a map?”

Main features

- Display maps and map tiles.
- Add markers for locations.
- Zoom, pan and change map type.
- Show the user's current location.
- Draw polylines and overlays.
- Respond to map clicks and marker interactions.

Real-world use cases

A delivery application can let users pin their home address on a map. A ride-hailing app can show a passenger's pickup point and the driver's position. A college application can display campus buildings and nearby facilities.

Implementation

Step 1 — Configure Google Maps

1. Create or select a project in Google Cloud.
2. Enable the Maps SDK for Android.
3. Configure billing as required by Google.
4. Create an API key and restrict it to the Android application and appropriate API.

5. Configure the key according to the official setup instructions.

Do not publish unrestricted API keys in source repositories.

Step 2 — Add the Maps dependency

In build.gradle.kts:

```
dependencies {
  implementation("com.google.android.gms:play-services-maps:<version>")
}
```

Step 3 — Add the map to the layout

In res/layout/activity_maps.xml:

```
<fragment
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/map_container"
  android:name="com.google.android.gms.maps.SupportMapFragment"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:context=".MapsActivity" />
```

Step 4 — Add a marker

In MapsActivity.java:

```
public class MapsActivity extends AppCompatActivity
      implements OnMapReadyCallback {

  private GoogleMap googleMap;

  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_maps);
```

```
    SupportMapFragment fragment =
        (SupportMapFragment) getSupportFragmentManager()
            .findFragmentById(R.id.map_container);

    if (fragment != null) {
        fragment.getMapAsync(this);
    }
  }

  @Override
  public void onMapReady(GoogleMap map) {
    googleMap = map;

    LatLng campus = new LatLng(12.0132, 79.8560);

    googleMap.addMarker(
        new MarkerOptions()
            .position(campus)
            .title("Campus")
    );

    googleMap.moveCamera(
        CameraUpdateFactory.newLatLngZoom(campus, 15f)
    );
  }
}
```

The coordinates above are an illustrative example; replace them with your actual campus coordinates.

How the code works

- SupportMapFragment provides the map container.
- getMapAsync(this) registers a callback that runs when the map is ready.
- onMapReady() receives the map object.
- LatLng represents a coordinate pair.
- addMarker() places a marker on the map.
- newLatLngZoom() moves the camera to the selected location and zoom level.

To display the blue current-location indicator, request and verify runtime location permission first, then enable the map's My Location layer. A map marker and the user's current location are different concepts.

1.5 Geocoding and reverse geocoding

Geocoding converts a human-readable address into geographical coordinates. Reverse geocoding converts coordinates into a human-readable address.

For example:

- Address to coordinates: Puducherry Technological University → latitude and longitude.
- Coordinates to address: latitude and longitude → a nearby street or locality.

Why is it needed?

Users generally prefer entering or viewing addresses instead of remembering numerical coordinates. Delivery services, navigation apps and location search tools use geocoding to connect addresses with map positions.

Java example

```
Geocoder geocoder = new Geocoder(this, Locale.getDefault());


try {
  List<Address> results = geocoder.getFromLocationName(
    "Chennai, India", 1);
```

```
    if (results != null && !results.isEmpty()) {
        Address address = results.get(0);

        double latitude = address.getLatitude();
        double longitude = address.getLongitude();

        Log.d("Geocoding", latitude + ", " + longitude);
    }
} catch (IOException e) {
    Log.e("Geocoding", "Address lookup failed", e);
}
```

Where to declare it: In an Activity or a suitable repository/service class. Run potentially blocking geocoding work off the main thread and handle unavailable results.

The Android Geocoder API may use a backend service and does not guarantee a result. For reliable production address lookup, check the selected geocoding provider's availability, quotas and usage terms.

1.6 Location updates and tracking

Some applications need more than one location reading. They require repeated updates as a device moves.

Examples include:

- Tracking a running route.
- Showing a delivery driver's changing position.
- Recording a cycling journey.
- Updating a vehicle's position on a map.

The fused location provider supports this using requestLocationUpdates().

A typical implementation involves:

1. Obtain the location client.
2. Create a LocationRequest specifying the required accuracy and interval.
3. Create a LocationCallback to receive results.

4. Check location permissions.
5. Register for updates while the feature is active.
6. Remove updates when they are no longer needed.

Illustrative Java setup:

```
LocationRequest request =
  new LocationRequest.Builder(
    Priority.PRIORITY_HIGH_ACCURACY, 5000
  ).setMinUpdateIntervalMillis(2500).build();


LocationCallback callback = new LocationCallback() {
  @Override
  public void onLocationResult(LocationResult result) {
    if (result == null) return;


    for (Location location : result.getLocations()) {
      Log.d("Tracking",
        location.getLatitude() + ", "
        + location.getLongitude());
    }
  }
};
```

After checking permissions, register with the fused location client using requestLocationUpdates(request, callback, Looper.getMainLooper()). Remove the callback using removeLocationUpdates(callback) when tracking ends.

The intervals are requests, not guaranteed update schedules. Background tracking requires additional consideration of foreground services, Android background-location rules and Google Play policies.

1.7 Location services versus Google Maps

Feature	Location services	Google Maps SDK

Main purpose	Determine geographical position	Display and interact with a map
Output	Location coordinates and updates	Visual map, markers and map interactions
Example API	FusedLocationProviderClient	GoogleMap, SupportMapFragment
Permissions	Location permissions when accessing device location	Maps configuration; location permission only for protected location features
Example	Obtain the device's latitude	Show a marker at that latitude

Classroom activity: Ask students to design a food-delivery app that detects the customer's position, allows the customer to adjust the address pin and displays the delivery location on a map.

Topic 2. Multimedia — Camera, Audio, Video and MediaPlayer

2.1 Introduction to multimedia

Definition: Multimedia integration is the use of different media types—such as images, sound and video—within a mobile application.

Android applications can capture media through device hardware, play media using system APIs and display media inside the user interface.

Why is multimedia needed?

Text alone is insufficient for many modern applications. A user may need to capture a document, record a voice message, watch a tutorial or listen to music.

Multimedia makes applications more interactive, informative and useful.

Real-world examples

- WhatsApp: photographs, voice notes and videos.
- Instagram: image capture, filters and video recording.
- Spotify: audio streaming and playback.
- YouTube: video playback.
- Google Lens: camera-based image analysis.
- Online learning apps: recorded lectures and demonstrations.

2.2 Camera integration

What is a camera API?

Camera APIs allow an Android application to capture photographs or videos using the device's camera.

Android provides two common approaches:

1. Camera intent: Opens another camera application to capture an image.
2. CameraX: Provides a higher-level API for creating a custom camera interface inside your app.

Camera intent versus CameraX

Feature	Camera intent	CameraX
Implementation	Simple	More involved
User interface	External camera app	Custom in-app interface
Control over camera	Limited	Greater
Suitable for	Basic photo capture	Scanning, preview and custom capture
Example	Select a profile photo	Build a document scanner

Use case: Profile picture capture

A student registration app lets a user take a photograph and immediately preview it on the profile screen.

Step 1 — Declare camera requirements

In AndroidManifest.xml:

<uses-feature

android:name="android.hardware.camera.any"

android:required="false" />

Setting required="false" allows devices without a camera to install the app. If a camera is essential to the app's core functionality, it may be appropriate to set it to true.

When delegating capture to another camera app using an intent, the app typically does not need to request direct camera permission itself. An application that accesses the camera hardware directly must handle the applicable camera permission.

Step 2 — Create the layout

In activity_camera.xml, add an ImageView with ID imageView and a button with ID btnCamera.

Step 3 — Launch the camera using Java

In CameraActivity.java:

```
public class CameraActivity extends AppCompatActivity {

  private ImageView imageView;

  private final ActivityResultLauncher<Void> cameraLauncher =
    registerForActivityResult(
      new ActivityResultContracts.TakePicturePreview(),
      bitmap -> {
        if (bitmap != null) {
          imageView.setImageBitmap(bitmap);
        }
      }
    );

  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_camera);

    imageView = findViewById(R.id.imageView);

    findViewById(R.id.btnCamera).setOnClickListener(v ->
      cameraLauncher.launch(null)
    );
```

```
    }
}
```

Required imports include ImageView, Bitmap, AppCompatActivity, ActivityResultLauncher and ActivityResultContracts.

Important limitation: TakePicturePreview() returns a small preview bitmap, not a guaranteed full-resolution photograph. For full-resolution capture, use ActivityResultContracts.TakePicture() with a content Uri pointing to a suitable output file.

How the code works

- TakePicturePreview() launches a camera app using an Android activity-result contract.
- cameraLauncher.launch(null) starts the capture flow.
- The callback receives a bitmap if the capture succeeds.
- setImageBitmap() displays that bitmap in the image view.

2.3 Audio recording

What is audio recording?

Audio recording captures sound using a microphone and stores it in a file or processes it for another purpose.

Use cases: Voice messages, audio diaries, lecture recording, speech-to-text systems and sound-analysis applications.

Android provides MediaRecorder for recording audio and video. Microphone access requires the RECORD_AUDIO permission and runtime permission approval.

Manifest declaration:

```
<uses-permission android:name="android.permission.RECORD_AUDIO" />
```

Simplified recording setup in Java, assuming the permission has already been granted and outputFile points to a writable app-owned file:

```
MediaRecorder recorder =
    new MediaRecorder(this);

recorder.setAudioSource(
    MediaRecorder.AudioSource.MIC);
```

```
recorder.setOutputFormat(
   MediaRecorder.OutputFormat.MPEG_4);

recorder.setAudioEncoder(
   MediaRecorder.AudioEncoder.AAC);

recorder.setOutputFile(outputFile);

recorder.prepare();
recorder.start();
```

To stop and release the recorder:

```
recorder.stop();
recorder.release();
recorder = null;
```

Important: The application must manage recorder state carefully. Calling stop() before valid audio has been recorded may fail. Production implementations should handle exceptions, clean up resources and stop recording appropriately when the activity or recording session ends.

2.4 Audio playback using MediaPlayer

What is MediaPlayer?

MediaPlayer is an Android framework class used to play audio and supported video sources. It can play local resources and supported media streams.

Why is it needed?

Applications need to play music, sound effects, notifications, recorded voice messages and other audio content.

Example: Play an audio file

First, create the folder res/raw/ and place a file named sample_audio.mp3 inside it.

In the layout, add a button with ID btnPlay.

In AudioActivity.java:

```
public class AudioActivity extends AppCompatActivity {

  private MediaPlayer player;

  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_audio);

    findViewById(R.id.btnPlay).setOnClickListener(v -> {
      releasePlayer();

      player = MediaPlayer.create(
        this, R.raw.sample_audio);

      if (player != null) {
        player.setOnCompletionListener(mp ->
          releasePlayer()
        );
        player.start();
      }
    });
  }

  private void releasePlayer() {
    if (player != null) {
      player.release();
```

```
        player = null;
    }
  }

  @Override
  protected void onStop() {
    super.onStop();
    releasePlayer();
  }
}
```

Explanation

- MediaPlayer.create() initializes a player for the resource.
- R.raw.sample_audio identifies the file in res/raw/.
- start() begins playback.
- setOnCompletionListener() allows the app to respond when playback finishes.
- release() frees resources when playback is no longer needed.

In more complex apps, manage pause/resume behaviour and audio focus, and avoid unexpectedly interrupting other applications' audio.

2.5 Video playback

Video playback combines moving images and audio, often with controls for pausing, seeking and changing playback position.

Common Android options

- VideoView: A straightforward option for simple embedded video playback.
- AndroidX Media3 ExoPlayer: A flexible option for streaming, adaptive playback and advanced media features.

For modern streaming applications, Media3 ExoPlayer is generally more appropriate than relying on a basic video view alone.

Example: VideoView

Add a VideoView with ID videoView in activity_video.xml.

```
VideoView videoView = findViewById(R.id.videoView);


Uri videoUri = Uri.parse(
   "android.resource://" + getPackageName()
   + "/" + R.raw.sample_video
);


videoView.setVideoURI(videoUri);
videoView.setMediaController(
   new MediaController(this));


videoView.getMediaController()
   .setAnchorView(videoView);


videoView.start();
```

Place a supported video file named sample_video inside res/raw/.

Explanation: setVideoURI() specifies the media source. MediaController provides basic playback controls. start() begins playback.

For production applications, ensure video playback respects lifecycle events and avoid automatically starting sound when it is inappropriate.

2.6 Multimedia lifecycle and resource management

Multimedia resources can consume memory, processing power and battery. They must be managed when the application becomes inactive.

Event	Recommended action
Playback begins	Prepare the player and start playback
User pauses playback	Pause the player
Playback finishes	Release or reuse the player appropriately
Activity is no longer visible	Stop or pause according to the app's design

Recording ends	Stop and release the recorder

Real-time case study: A student watches a recorded lecture. A phone call arrives, the lecture pauses, and after the call the student resumes playback from the correct position. This requires coordinated media state, audio focus and lifecycle management.

Topic 3. Sensors — Accelerometer, Gyroscope and Other Sensors

3.1 Introduction to sensors

Definition: A sensor is a hardware component or software-based sensing mechanism that detects physical conditions or changes and makes the corresponding measurements available to an application.

Smartphones contain sensors that can detect movement, rotation, proximity, light and magnetic fields.

Why are sensors needed?

Touchscreens detect direct user interaction, but sensors enable an application to respond to how a user moves or holds the device and to conditions in the surrounding environment.

Real-world applications

- Google Fit and fitness applications: activity and movement measurements.
- Racing games: tilt-based steering.
- AR applications: device movement and orientation.
- Phone calls: proximity-based screen control.
- Camera applications: orientation and stabilization support.
- Automatic screen brightness: ambient light sensing.

3.2 Android Sensor Framework

Android provides a sensor framework that allows applications to discover available sensors, register listeners and receive measurements.

The principal components are:

Component	Purpose
SensorManager	Accesses and manages device sensors
Sensor	Represents a particular sensor

SensorEventListener	Receives sensor readings
SensorEvent	Contains a reading, timestamp and sensor information
registerListener()	Starts receiving readings
unregisterListener()	Stops receiving readings

How it works

Physical movement or environmental change

Device sensor measures the change

Android Sensor Framework

onSensorChanged() receives measurements

Application updates its behaviour or interface

3.3 Accelerometer

What is an accelerometer?

An accelerometer measures acceleration along the device's three coordinate axes: X, Y and Z. Its measurements include the effect of gravity.

The unit is metres per second squared (m/s^2).

- X-axis: side-to-side direction of the device.
- Y-axis: bottom-to-top direction of the device.
- Z-axis: perpendicular to the screen.

The axes refer to the device's coordinate system, not necessarily the user's left and right when the phone is rotated.

Why is it needed?

The accelerometer allows applications to detect movement, tilting and sudden changes in acceleration.

Use cases

1. Detecting a shake gesture.
2. Measuring movement during exercise.
3. Controlling games by tilting a phone.
4. Supporting fall-detection algorithms.

5. Estimating movement patterns.

Important: An accelerometer alone cannot reliably count every step or distinguish every fall. Robust applications combine appropriate algorithms and, where needed, additional sensor information.

3.4 Gyroscope

What is a gyroscope?

A gyroscope measures angular velocity: the rate at which a device rotates around its X, Y and Z axes.

The unit is radians per second (\(\text{rad/s}\)).

Why is it needed?

The gyroscope helps applications estimate rotational movement and changes in orientation. It complements accelerometer data, which measures acceleration rather than angular velocity.

Use cases

- Steering in racing games.
- Motion control in virtual reality.
- Rotational tracking in augmented reality.
- Camera-motion estimation.
- Smooth movement in immersive 360-degree experiences.

Accelerometer versus gyroscope

Feature	Accelerometer	Gyroscope
Measures	Acceleration, including gravity	Angular velocity
Unit	\(\text{m/s}^2\)	\(\text{rad/s}\)
Typical use	Detect movement or tilt	Detect rotation
Limitation	Movement and gravity can be difficult to separate	Integrated rotation estimates can drift
Example	Detecting a shake	Detecting a turn

3.5 Other important sensors

Proximity sensor

Detects whether an object is near the device. A common use is turning off the screen when a phone is held against the user's face during a call.

Ambient light sensor

Measures surrounding light intensity. A phone may use it to adjust screen brightness automatically.

Magnetic field sensor

Measures the magnetic field around the device. It can help a compass determine direction when combined with suitable calibration and other sensor data.

Barometer

Measures atmospheric pressure. Some devices use it to help estimate relative altitude or elevation changes.

Not every Android device contains every sensor. Applications should check whether a required sensor is available.

3.6 Java implementation: Read accelerometer values

Step 1: Create activity_sensor.xml with a TextView whose ID is txtSensor.

Step 2: Create SensorActivity.java.

```
public class SensorActivity extends AppCompatActivity
    implements SensorEventListener {

  private SensorManager sensorManager;
  private Sensor accelerometer;
  private TextView txtSensor;

  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_sensor);
```

```
    txtSensor = findViewById(R.id.txtSensor);

    sensorManager = (SensorManager)
        getSystemService(Context.SENSOR_SERVICE);

    if (sensorManager != null) {
        accelerometer = sensorManager.getDefaultSensor(
            Sensor.TYPE_ACCELEROMETER);
    }

    if (accelerometer == null) {
        txtSensor.setText("Accelerometer unavailable");
    }
}

@Override
protected void onResume() {
    super.onResume();

    if (accelerometer != null) {
        sensorManager.registerListener(
            this, accelerometer,
            SensorManager.SENSOR_DELAY_NORMAL);
    }
}

@Override
protected void onPause() {
```

```
    super.onPause();

    if (sensorManager != null) {
        sensorManager.unregisterListener(this);
    }
}

@Override
public void onSensorChanged(SensorEvent event) {
    if (event.sensor.getType() ==
            Sensor.TYPE_ACCELEROMETER) {

        float x = event.values[0];
        float y = event.values[1];
        float z = event.values[2];

        txtSensor.setText(String.format(
            Locale.getDefault(),
            "X: %.2f\nY: %.2f\nZ: %.2f",
            x, y, z));
    }
}

@Override
public void onAccuracyChanged(
        Sensor sensor, int accuracy) {
    // Handle accuracy changes if needed.
}
```

}

Required imports include Context, Sensor, SensorEvent, SensorEventListener, SensorManager, TextView, Locale and AppCompatActivity.

Explanation

- getSystemService() obtains Android's sensor manager.
- getDefaultSensor() retrieves the requested sensor if available.
- registerListener() starts receiving sensor events.
- event.values contains the sensor measurements.
- unregisterListener() stops updates when the activity is paused.

For high-frequency sensor processing, use the appropriate sampling rate, process events efficiently and avoid doing expensive operations inside onSensorChanged().

3.7 Mini-project: Shake-to-change-screen app

A simple application can change its background when the phone is shaken.

Basic logic:

1. Read the accelerometer's X, Y and Z values.
2. Calculate the magnitude of the measured acceleration.
3. Compare it with a chosen threshold.
4. Detect a sufficiently strong movement.
5. Change the background or display a message.

An illustrative magnitude calculation is:

```
double magnitude = Math.sqrt(
  x * x + y * y + z * z);
```

Because gravity contributes to the accelerometer readings, a real shake detector should account for the baseline, apply suitable filtering and avoid triggering repeatedly for the same movement.

Topic 4. Firebase — Authentication, Realtime Database, Cloud Messaging and Cloud Storage

4.1 Introduction to Firebase

Definition: Firebase is a collection of cloud-based application development services provided by Google. It helps developers build mobile and web applications using managed infrastructure and SDKs.

Firebase can provide identity management, data storage, file storage, notifications and other backend capabilities.

Why is Firebase needed?

A mobile app often needs to store data centrally, identify users, share updates across devices and notify users when something happens.

Without a managed backend, developers would need to build and maintain more of this infrastructure themselves.

Real-world examples

- Chat applications: authentication and real-time messaging data.
- Shopping applications: user accounts and profile information.
- Learning platforms: storing course progress and notifying students.
- Photo-sharing applications: uploading profile images.
- Collaborative applications: synchronizing updates across connected devices.

4.2 Firebase architecture

Android application

Java UI and Firebase SDKs

Firebase services

Authentication · Realtime Database · Cloud Messaging · Cloud Storage

Cloud infrastructure and security rules

Identity, stored data, uploaded files and notification delivery

Different Firebase services have different data flows. For example, Cloud Messaging delivers messages to client devices, whereas Authentication verifies identities and the database stores application data.

4.3 Firebase project setup

Before using Firebase in Android Studio:

1. Create a project in the Firebase Console.
2. Register your Android app using its application ID.

3. Download google-services.json.
4. Place the configuration file in the app module directory.
5. Configure the Google Services Gradle plugin according to the current Firebase setup guide.
6. Add the dependencies for the services you plan to use.
7. Configure authentication providers, database rules, storage rules or messaging as needed.

Use compatible current dependency versions rather than copying old tutorial versions blindly.

4.4 Firebase Authentication

What is authentication?

Authentication verifies the identity of a user attempting to access an application.

Examples include email-and-password login, phone authentication and supported federated sign-in providers.

Why is it needed?

A shopping app must distinguish one user's orders from another user's orders. A learning platform may need to identify which student completed a lesson.

Authentication establishes the user's identity; authorization determines what that user is allowed to access.

Use cases

- Registration and login.
- Password reset.
- User account management.
- Signed-in sessions.
- Access control when combined with database and storage security rules.

Java example: Email and password sign-in

Add the Firebase Authentication dependency using the current Firebase BoM and configuration.

```
dependencies {
  implementation(platform(
```

```
    "com.google.firebase:firebase-bom:<current-version>"
  ))
  implementation("com.google.firebase:firebase-auth")
}
```

In the Activity, after the Firebase project is configured and the user has entered credentials into the UI:

```
private FirebaseAuth auth;

@Override
protected void onCreate(Bundle savedInstanceState) {
  super.onCreate(savedInstanceState);
  setContentView(R.layout.activity_login);

  auth = FirebaseAuth.getInstance();
}
```

Then create a sign-in method:

```
private void signIn(String email, String password) {
  auth.signInWithEmailAndPassword(email, password)
    .addOnCompleteListener(this, task -> {
      if (task.isSuccessful()) {
        FirebaseUser user = auth.getCurrentUser();

        if (user != null) {
          String uid = user.getUid();
          // Open the authenticated user's screen.
        }
      } else {
        Toast.makeText(
```

```
                this,
                "Sign-in failed. Check your credentials.",
                Toast.LENGTH_SHORT
            ).show();
        }
    });
}
```

Call signIn(email, password) from the login button after validating the input fields.

Explanation

- FirebaseAuth.getInstance() obtains the Firebase Authentication instance.
- signInWithEmailAndPassword() sends the sign-in request.
- task.isSuccessful() indicates whether authentication succeeded.
- getCurrentUser() retrieves the signed-in user.
- getUid() returns the user's unique identifier.

For registration, use createUserWithEmailAndPassword(). Passwords should never be hard-coded or stored as plain text by the app. Enable the required sign-in provider in the Firebase Console and implement appropriate error handling.

4.5 Firebase Realtime Database

What is it?

Firebase Realtime Database is a cloud-hosted NoSQL database that stores data as a JSON tree and synchronizes changes to connected clients.

NoSQL means the database does not rely on the traditional relational table model in the same way as a relational database.

Why is it needed?

When data changes, connected applications can receive updates without repeatedly polling a server.

Real-world use cases

- Live chat messages.
- Real-time classroom polls.

- Collaborative task lists.
- Live status updates.
- Simple synchronized dashboards.

Example data structure

A database might contain:

```
{
  "students": {
    "student001": {
      "name": "Student A",
      "score": 85
    }
  }
}
```

The JSON structure illustrates a possible data model. Actual production data should use the real authenticated user's identifier and appropriate access rules.

Java example: Write and listen for data

Add the Realtime Database dependency.

```
implementation("com.google.firebase:firebase-database")
```

In a Java Activity:

```
DatabaseReference scoreRef =
    FirebaseDatabase.getInstance()
        .getReference("classroom/score");


private void saveScore(int score) {
    scoreRef.setValue(score);
}
```

To listen for updates:

```
scoreRef.addValueEventListener(
```

```
  new ValueEventListener() {

    @Override
    public void onDataChange(DataSnapshot snapshot) {
      Integer score = snapshot.getValue(Integer.class);

      if (score != null) {
        Log.d("Classroom", "Score: " + score);
      }
    }

    @Override
    public void onCancelled(DatabaseError error) {
      Log.e("Classroom", "Read failed",
          error.toException());
    }
  }
);
```

Explanation

- getReference() selects a database path.
- setValue() writes a value at that path.
- addValueEventListener() registers a listener.
- onDataChange() runs when the observed data changes.
- onCancelled() handles a failed or denied operation.

For production apps, configure Realtime Database Security Rules so users can only access the data they are authorized to read or change. Remove listeners when they are no longer needed.

4.6 Firebase Cloud Messaging (FCM)

What is it?

Firebase Cloud Messaging is a service for delivering messages and push notifications to supported client applications.

Why is it needed?

Applications often need to inform users about events even when the application is not currently open.

Use cases

- Delivery status updates.
- New chat message notifications.
- Class announcements.
- Appointment reminders.
- Account-related alerts.

How it works

1. The app integrates the FCM SDK.
2. The client obtains an FCM registration token when appropriate.
3. A trusted sender, commonly an application server or Cloud Functions workflow, requests delivery through FCM.
4. FCM routes the message to the target device.
5. Android and the application handle the message according to its type and the app's state.

The registration token identifies an app installation for messaging purposes; it is not proof of a user's identity.

Java example: Receive messages

Add the Messaging dependency:

implementation("com.google.firebase:firebase-messaging")

Create a Java class:

```
public class MyFirebaseMessagingService
    extends FirebaseMessagingService {


  @Override
```

```
  public void onMessageReceived(
      RemoteMessage message) {
    super.onMessageReceived(message);

    if (message.getNotification() != null) {
      Log.d("FCM",
        message.getNotification().getBody());
    }

    if (!message.getData().isEmpty()) {
      Log.d("FCM",
        message.getData().toString());
    }
  }

  @Override
  public void onNewToken(String token) {
    Log.d("FCM", "Token refreshed");
    // Send the token securely to your backend if needed.
  }
}
```

Register the service inside the <application> element in AndroidManifest.xml:

```
<service
  android:name=".MyFirebaseMessagingService"
  android:exported="false">
  <intent-filter>
    <action android:name=
      "com.google.firebase.MESSAGING_EVENT" />
```

```
    </intent-filter>
</service>
```

Important distinction

- Notification messages: Usually handled by the Android system tray when the app is in the background.
- Data messages: Delivered to application code under the applicable conditions and can be used for custom handling.
- Foreground delivery: The application can process incoming messages while it is active, but it must decide how to present them.

On Android 13 and later, apps generally need to request notification permission before showing ordinary notifications. High-priority messaging should be used only for appropriate time-sensitive work.

4.7 Firebase Cloud Storage

What is it?

Firebase Cloud Storage stores user-generated files, including photographs, videos and documents.

It is designed for binary objects rather than representing the entire file as a database record.

Why is it needed?

A user profile image or recorded lecture is usually better stored as a file in cloud storage. A database can then store the file's reference or download URL alongside the relevant metadata.

Real-world use cases

- Profile picture uploads.
- Assignment submissions.
- Product images.
- Shared photographs.
- Recorded audio and video.

Java example: Upload an image

Add the Storage dependency:

```
implementation("com.google.firebase:firebase-storage")
```

Assuming the user has selected an image and imageUri is a valid content URI:

```
StorageReference storageRef =
  FirebaseStorage.getInstance()
    .getReference("profile-images/" + userId + ".jpg");


storageRef.putFile(imageUri)
  .addOnSuccessListener(taskSnapshot ->
    storageRef.getDownloadUrl()
      .addOnSuccessListener(uri -> {
        String downloadUrl = uri.toString();
        // Save the URL in the appropriate database record.
      })
  )
  .addOnFailureListener(error ->
    Log.e("Storage", "Upload failed", error)
  );
```

Explanation

- getReference() identifies the destination path.
- putFile() uploads the image.
- getDownloadUrl() retrieves a URL that can be used to access the file, subject to the storage access configuration.
- The URL or file path can be saved as metadata in a database.

Do not allow unrestricted uploads or public access by default. Use authenticated access, Firebase Storage Rules, file-size limits and appropriate content validation.

4.8 Comparing the four Firebase services

Service	Main responsibility	Example
Authentication	Verify user identity	Login
Realtime Database	Store and synchronize structured data	Live score

Cloud Messaging	Deliver messages and notifications	Order update
Cloud Storage	Store files	Profile photograph

Topic 5. Integration with Third-Party APIs — Social Media and Payment Gateways

5.1 Introduction to third-party API integration

Definition: Third-party API integration is the process of connecting an application to a service developed and maintained by another organization through a defined interface.

An API (Application Programming Interface) specifies how software components request data or actions from one another.

Why is it needed?

Developers do not need to implement every capability from scratch. They can integrate existing services for maps, payments, social sharing, authentication, messaging, weather information and many other functions.

Real-world examples

- A shopping app retrieves product information from a backend API.
- A travel app requests weather information from a weather API.
- An e-commerce app integrates a payment provider.
- A news app allows users to share articles.
- A mobile app uses a supported identity provider to sign users in.

5.2 What is a REST API?

REST stands for Representational State Transfer. A REST-style web API exposes resources through HTTP endpoints and uses HTTP methods to perform operations.

Common methods include:

Method	Typical purpose	Example
GET	Retrieve data	Fetch products
POST	Submit data or create a resource	Place an order
PUT	Replace a resource	Update a profile

PATCH	Partially update a resource	Change a delivery address
DELETE	Remove a resource	Delete a saved item

These are common conventions; the actual behaviour depends on the API contract.

Example request and response

A mobile application might request:

GET /products/101

The server could return:

```
{
  "id": 101,
  "name": "Wireless Headphones",
  "price": 1999
}
```

The Android application parses the response and displays the product information.

Architecture

Android UI

User taps "Load Products"

Networking layer

Retrofit / OkHttp or another HTTP client

Backend API

Validates request and retrieves data

JSON response

Android parses the data and updates the UI

5.3 Making a REST API request in Android using Java

A networking library such as Retrofit can simplify HTTP requests. For a teaching example, Android's built-in HttpURLConnection can also illustrate the underlying process.

Step 1 — Add internet permission

Inside the <manifest> element in AndroidManifest.xml:

<uses-permission android:name="android.permission.INTERNET" />

Step 2 — Perform a GET request

For demonstration, the following example requests public sample JSON from JSONPlaceholder. It runs on a background thread and logs the response.

```
private void fetchPost() {
  ExecutorService executor =
    Executors.newSingleThreadExecutor();

  executor.execute(() -> {
    HttpURLConnection connection = null;

    try {
      URL url = new URL(
        "https://jsonplaceholder.typicode.com/posts/1");

      connection = (HttpURLConnection)
        url.openConnection();

      connection.setRequestMethod("GET");
      connection.setConnectTimeout(10000);
      connection.setReadTimeout(10000);

      int status = connection.getResponseCode();

      InputStream stream = status < 400
        ? connection.getInputStream()
        : connection.getErrorStream();
```

```
        ByteArrayOutputStream output =
            new ByteArrayOutputStream();

        byte[] buffer = new byte[4096];
        int count;

        while ((count = stream.read(buffer)) != -1) {
            output.write(buffer, 0, count);
        }

        String response = output.toString("UTF-8");

        runOnUiThread(() ->
            Log.d("API", "HTTP " + status + ": " + response)
        );

      } catch (Exception e) {
        Log.e("API", "Request failed", e);
      } finally {
        if (connection != null) {
          connection.disconnect();
        }
        executor.shutdown();
      }
    });
}
```

Required imports include URL, HttpURLConnection, InputStream, ByteArrayOutputStream, ExecutorService and Executors.

Explanation

- URL represents the endpoint.
- openConnection() creates the HTTP connection.
- setRequestMethod("GET") specifies the request method.
- getResponseCode() retrieves the HTTP status.
- The response stream contains the returned data.
- runOnUiThread() moves UI-related work back to the main thread.

This example logs the response rather than displaying it in a TextView. In a real application, parse the JSON, update the interface, show loading and error states, and cancel unnecessary work when appropriate. A production implementation would generally use a reusable networking layer.

5.4 Social media integration

Social media integration allows users to share content or connect an app to supported social platform features.

Two main approaches

Approach 1: Android Sharesheet

The application asks Android to display available apps through which the user can share text, a link or other supported content.

Advantages:

- Simple to implement.
- Does not require a dedicated social media SDK.
- Lets users select their preferred application.
- Avoids handling third-party login credentials.

Approach 2: Official provider SDK or API

Use this when the application requires supported platform-specific functionality, such as an approved sign-in flow or access to particular account features.

This approach may require a developer account, app registration, consent and platform-specific permissions.

Java example: Share a link

```
private void shareContent() {
```

```
    Intent sendIntent = new Intent(Intent.ACTION_SEND);

    sendIntent.setType("text/plain");

    sendIntent.putExtra(
        Intent.EXTRA_TEXT,
        "Explore this learning resource: https://example.com"
    );

    Intent chooser = Intent.createChooser(
        sendIntent, "Share using");

    startActivity(chooser);
}
```

Call shareContent() from a button click listener.

Explanation

- ACTION_SEND requests a sharing action.
- setType("text/plain") specifies the type of content.
- EXTRA_TEXT contains the message.
- Intent.createChooser() lets the user select a compatible application.

This does not guarantee that a particular social media application is installed. Android displays compatible sharing targets.

5.6 Payment gateway integration

What is a payment gateway?

A payment gateway or payment service provider enables an application to initiate and process payment transactions through supported payment methods.

Examples of providers and services include Razorpay, Stripe and Google Pay, although their availability, supported payment methods and integration models vary by region and product.

Why is it needed?

An online shopping application needs a secure way to accept payment, verify transaction status and associate successful payments with orders.

Real-world use cases

- E-commerce checkout.
- Ticket booking.
- Food delivery.
- Subscription services.
- Online course purchases.
- Utility bill payments.

Payment architecture

1. Android application

User chooses an item and initiates checkout

2. Trusted backend

Creates an order and requests a payment session

3. Payment provider

Processes payment using the supported payment flow

4. Backend verification

Verifies the result and confirms the order

Why is the backend needed?

The mobile application runs on a device controlled by the user. A malicious user could modify the app, change local data or simulate a success screen.

Consequently, the app must not be the final authority on whether an order is paid.

The backend should:

1. Calculate or validate the amount using trusted order data.
2. Create a payment order or session with the provider.
3. Use secret credentials only on the server.
4. Receive and verify provider notifications or webhooks.
5. Confirm payment status before fulfilling the order.

6. Handle duplicate events, retries and failed transactions safely.

Important security principles

- Never embed secret API keys in the Android application.
- Do not store raw payment card data unless a specifically designed, compliant solution permits it.
- Use provider-supported SDKs and secure payment flows.
- Verify transaction signatures and server-side status.
- Do not trust a client-side success message alone.
- Use HTTPS and follow the provider's current security and compliance requirements.

Topic 6. Accessibility in Mobile Applications

6.1 What is accessibility?

Definition: Accessibility is the practice of designing and developing applications so that people with different abilities can use them effectively.

Accessibility considers users with visual, hearing, motor, cognitive and other access needs.

It is not simply a visual design enhancement. It is an important part of making an application usable by more people.

Why is it needed?

An app may look attractive but still be difficult to use if its buttons have no accessible names, its text cannot be enlarged, or its controls depend only on colour.

Accessible design helps users understand the interface, operate controls and complete tasks independently.

Real-world examples

- TalkBack reads interface elements aloud.
- Voice input can help users interact without typing.
- Captions make video content more accessible.
- Large touch targets support users with motor difficulties.
- High-contrast text improves readability.
- Semantic labels explain images and controls to assistive technologies.

6.2 Screen readers and TalkBack

What is TalkBack?

TalkBack is Android's screen reader. It provides spoken feedback and supports interaction with interface elements using accessibility gestures.

A screen reader relies on information exposed by the application's UI, including accessible names, roles and states.

Example: An image with an accessible description

In XML:

```
<ImageView
  android:id="@+id/imgProfile"
  android:layout_width="120dp"
  android:layout_height="120dp"
  android:src="@drawable/profile_photo"
  android:contentDescription="Student profile photo" />
```

Explanation: contentDescription gives an image a meaningful accessible name. A screen reader can announce that name when the image is encountered.

If an image is purely decorative and adds no information, it should generally not create unnecessary screen-reader output. For such an image, use an appropriate decorative configuration, such as android:contentDescription="@null".

6.3 Accessible buttons and controls

A button should have a meaningful visible label and an appropriate accessibility role.

For example:

```
<Button
  android:id="@+id/btnSubmit"
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:minHeight="48dp"
  android:text="Submit assignment" />
```

This example gives the button a descriptive label and a minimum height of 48dp.

Avoid labels such as "Click here" when a more informative label would clarify the action. Custom controls should expose their purpose and state to assistive technologies.

6.4 Touch target sizes

A touch target is the interactive area a user must touch to activate a control.

Small controls can be difficult to select, particularly for users with motor impairments.

Android design guidance generally recommends a touch target of at least 48 × 48 dp for interactive elements.

Ways to improve touch usability include:

- Increase the interactive area.
- Leave sufficient spacing between adjacent controls.
- Avoid placing essential actions too close together.
- Make sure icons are recognisable and have accessible labels.
- Do not rely on precise gestures when a simpler control is possible.

The visible icon can be smaller than the actual touch target.

6.5 Colour contrast

Colour contrast describes the difference in luminance between foreground and background colours.

Poor contrast makes text and controls harder to perceive.

For typical web-style accessibility guidance based on WCAG, common minimum contrast ratios are:

- 4.5:1 for normal text.
- 3:1 for large text.
- 3:1 for important graphical objects and user-interface components in applicable cases.

These are useful accessibility benchmarks, but conformance should be assessed against the applicable standard and component type.

Good and poor design examples

Poor practice	Better practice

Light-grey text on a white background	Text with sufficient contrast
Red means failure and green means success, with no other indication	Use text or icons in addition to colour
Tiny icons with no labels	Adequately sized controls with accessible names
Fixed text sizes that clip content	Layouts that accommodate larger font settings

6.6 Dynamic text and screen scaling

Users may increase system font size to make content easier to read.

Applications should:

- Use scalable text units such as sp.
- Allow layouts to grow when text becomes larger.
- Avoid fixed-height text containers that cut off content.
- Check long labels, buttons and dialogs at larger font sizes.
- Preserve logical reading and focus order.

Example:

```
<TextView
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:text="Welcome to the application"
  android:textSize="18sp" />
```

Using wrap_content for the height and sp for text size helps the layout adapt to different text settings.

6.7 Keyboard navigation and focus

Some users navigate applications with external keyboards, switches or other assistive devices.

Good accessibility requires:

- A logical focus order.
- Clearly visible keyboard focus.

- Meaningful labels for input fields.
- Controls that can be operated without precise touch gestures.
- Appropriate focus handling when dialogs open or close.

A visually logical layout should also expose a logical sequence to assistive technologies.

6.8 Testing accessibility

Accessibility testing should include both automated tools and manual use.

A useful testing process is:

1. Enable TalkBack on an Android test device.
2. Navigate through the app using accessibility gestures.
3. Check whether each important control has a meaningful name.
4. Increase system font size and test for clipping.
5. Inspect contrast and touch target sizes.
6. Test with keyboard or switch navigation where relevant.
7. Use Android's accessibility testing tools to identify potential issues.

Important: A successful automated accessibility scan does not guarantee that an app is fully accessible. Manual testing is also necessary.

6.9 Accessibility mini-project

Build an accessible student registration screen with:

- Clearly labelled name and email fields.
- A submit button with an adequate touch target.
- Error messages that explain how to correct input.
- Readable text at larger system font settings.
- Appropriate labels for meaningful images.
- A logical focus order and sufficient colour contrast.